File: mL_algo_from_scratch/knn.py

Purpose: K-Nearest Neighbors Algorithm (Distance-based learning)

```
"""Simple K-Nearest Neighbors classifier/regressor written plainly.

This version uses straightforward loops and comments so it's easy to follow.
"""
import numpy as np
```

=== Classes ===

```
class KNN:
```

Study Tip: Initialization sets up the state. Look for hyperparameters.

Logic Focus: Defined task specifically for '__init__'

```
    def __init__(self, k=3, task="classification"):
        """k: number of neighbors; task: 'classification' or 'regression'"""
        self.k = int(k)
        self.task = task
        self.X_train = None
        self.y_train = None
```

Study Tip: This method usually trains the model. Check how dimensions are handled.

Logic Focus: Defined task specifically for 'fit'

```
    def fit(self, X, y):
        X = np.asarray(X)
        if X.ndim == 1:
            X = X.reshape(-1, 1)
        self.X_train = X
        self.y_train = np.asarray(y)
        return self
```

Memory Trick: Distance metrics are the core of KNN. Common ones: Euclidean, Manhattan.

Logic Focus: Defined task specifically for '_euclidean_distance'

```
    def _euclidean_distance(self, a, b):
```

Memory Trick: Distance metrics are the core of KNN. Common ones: Euclidean, Manhattan.

```
        """Compute Euclidean distance between two 1-D arrays."""
        return np.sqrt(np.sum((a - b) ** 2))
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

Logic Focus: Defined task specifically for 'predict'

```
    def predict(self, X):
        X = np.asarray(X)
        if X.ndim == 1:
            X = X.reshape(-1, 1)
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

```
        predictions = []
        for x in X:
```

Memory Trick: Distance metrics are the core of KNN. Common ones: Euclidean, Manhattan.

```
            # compute distance from x to every training example
```

Memory Trick: Distance metrics are the core of KNN. Common ones: Euclidean, Manhattan.

```
            distances = []
            for xt in self.X_train:
```

Memory Trick: Distance metrics are the core of KNN. Common ones: Euclidean, Manhattan.

```
                distances.append(self._euclidean_distance(x, xt))
```

Memory Trick: Distance metrics are the core of KNN. Common ones: Euclidean, Manhattan.

```
            # get indices of the k smallest distances
```

Memory Trick: Distance metrics are the core of KNN. Common ones: Euclidean, Manhattan.

```
            idx_sorted = np.argsort(distances)[: self.k]
            neighbor_labels = self.y_train[idx_sorted]

            if self.task == "classification":
                # majority vote (simple loop to be clear)
                counts = {}
                for lab in neighbor_labels:
                    counts[int(lab)] = counts.get(int(lab), 0) + 1
                # pick label with max count
                best = max(counts.items(), key=lambda t: t[1])[0]
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

```
                predictions.append(best)
            else:
                # regression: average of neighbor values
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

```
                predictions.append(float(np.mean(neighbor_labels)))
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

```
        return np.array(predictions)
```

Logic Focus: Defined task specifically for 'score'

```
    def score(self, X, y):
```

Key Logic: Inference happens here. Ensure data is preprocessed identically to training.

```
        preds = self.predict(X)
        if self.task == "classification":
            return float(np.mean(preds == y))
        else:
```

=== Imports ===

```
            from .utils import mse

            return mse(y, preds)
```